



Article development led by **acmqueue**
queue.acm.org

Designing an algorithm with reduced connection churn that could replace deterministic subsetting.

BY PETER WARD, PAUL WANKADIA, AND KAVITA GULIANI

Reinventing Backend Subsetting at Google

IN RECENT YEARS, the Autopilot^a system at Google has become increasingly popular internally for improving resource utilization. Autopilot can do multiple things: It can be configured to perform horizontal scaling, which adjusts the number of tasks a service has running to meet the demand; and it can be configured to perform vertical scaling, which adjusts the CPU/memory resources provisioned per task. Autopilot is also effective in preventing outages: It can respond to increased demand by scaling up a service faster than the human operators.

As the usage of Autopilot became widespread, service owners discovered an interesting problem: Whenever a horizontally scaled service resized, many

client connections (usually long-lived) would briefly drop and reconnect. This connection churn caused second-order effects:

- ▶ Increased errors or latency for in-flight requests
- ▶ Increased CPU/memory usage from connection handshakes
- ▶ Reduced throughput from TCP slow start^b on newly established connections
- ▶ Increased pressure on connection caches

The severity of these effects varied by service, but in some cases, the increased errors or latency put the services' service-level objectives at risk and blocked the adoption of Autopilot. Investigation determined that this connection churn was caused by backend subsetting.

Backend subsetting—a technique for reducing the number of connections when connecting services together—is useful for reducing costs and may even be necessary for operating within the system limits. For more than a decade, Google used deterministic subsetting^c as its default backend subsetting algorithm, but although this algorithm balances the number of connections per backend task, deterministic subsetting has a high level of connection churn.

Our goal at Google was to design an algorithm with reduced connection churn that could replace deterministic subsetting as the default backend subsetting algorithm. It was ambitious because, as Hyrum's Law^d states, "All observable behaviors of your system will be depended on by somebody." We needed to understand all the behaviors of deterministic subsetting to avoid regressions.

Backend Subsetting in Borg

Google services run on Borg,^e the company's cluster management software. Service owners configure *jobs* running in multiple Borg *cells* for geographical diversity. Within a cell, a job consists of one or more *tasks*, with each task run-

a <https://research.google/pubs/pub49174/>

b https://en.wikipedia.org/wiki/TCP_congestion_control#Slow_start

c <https://bit.ly/3IyyEhs>

d <https://www.hyrumslaw.com/>

e <https://research.google/pubs/pub43438/>



ning on some machine in the datacenter. The tasks are numbered consecutively from zero.

Backend subsetting is used when connecting jobs together—if a frontend job consisting of M tasks connects to a backend job consisting of N tasks, there would normally be $M \times N$ connections, which can be quite large when jobs have thousands of tasks. Instead, each of the M frontend tasks connects to k of the backend tasks, reducing the number of connections to $M \times k$. Choosing an appropriate value for k is left to the reader, but it will usually be much less than M or N .

To use backend subsetting, the service must be *replicated*: If the same request is sent to different tasks, they should perform equivalent work and return equivalent responses.

A load-balancing policy at the frontend task is used to direct each request to a specific backend task, with the

goal of uniform usage across backend tasks. Each backend task is allocated the same resources, so to avoid overload, we need to provision for the most loaded backend task.

The Previous Approach

The subsets chosen by the backend subsetting algorithm have various effects on production: connection balance, subset diversity, connection churn, and subset spread. To describe these behaviors and explain how the new algorithm was developed, let's start with a simple algorithm and improve it iteratively.

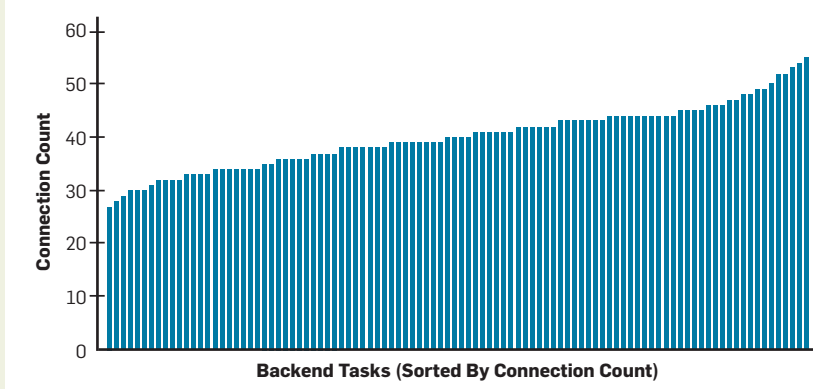
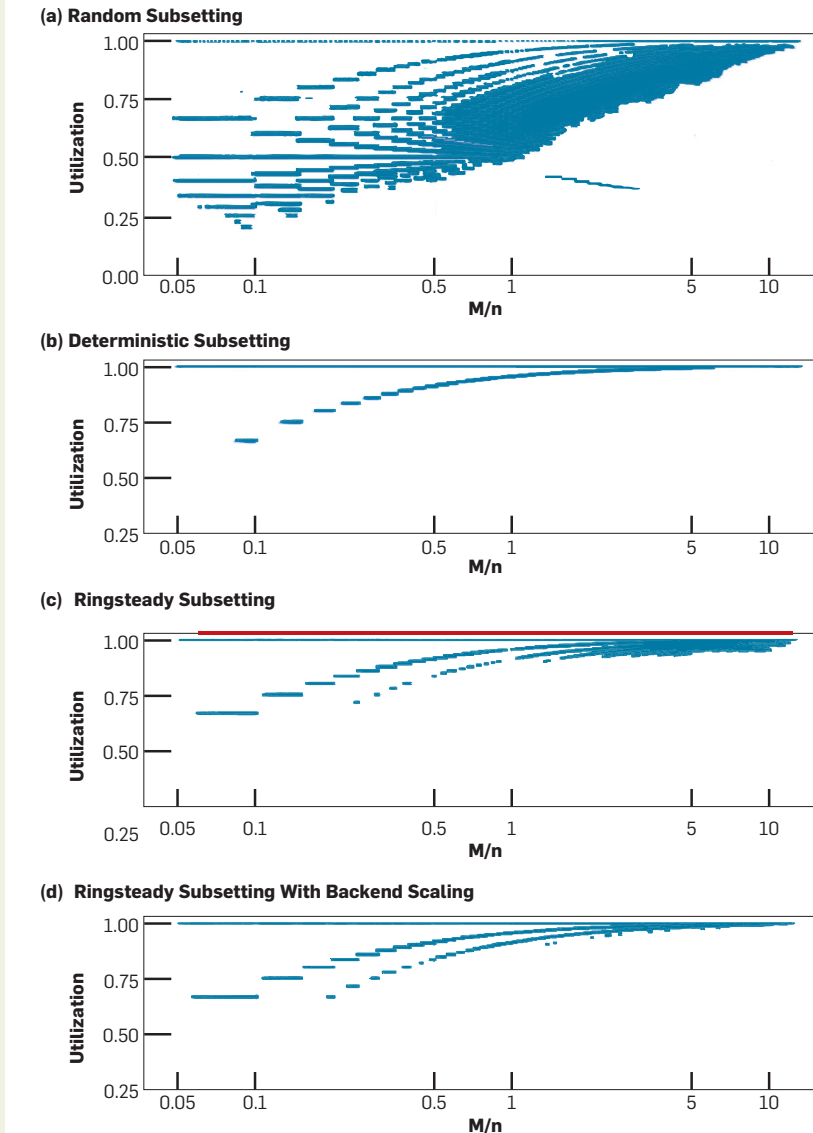
Random subsetting. One of the simplest possible algorithms is to choose random subsets: Each frontend task shuffles the list of backend tasks (identified by task numbers 0 to $N-1$) and selects the first k tasks.

Unfortunately, this interacts poorly with many load-balancing policies.

Suppose you have a CPU-bound service where all requests have the same cost and each frontend task uses round-robin load balancing to balance requests evenly across backend tasks. Thus, the load on each backend task would be directly correlated with the number of connections to it. The connection distribution from random subsetting is far from uniform, however, as Figure 1 shows.

Round robin is a simple load-balancing policy but not the only one influenced by the connection distribution. Given the diversity of Google services and their differing load-balancing requirements, requiring connection-agnostic load-balancing policies is impractical. Therefore, the subsetting algorithm should strive to balance the connection distribution.

Property: Connection balance. The goal is to measure the amount of load imbalance contributed by the subset-

Figure 1. Connection balance of random subsetting.**Figure 2. Connection balance of subsetting variety of scenarios.**

ting algorithm, assuming that the load-balancing policy is influenced by the connection distribution. To do this, every frontend task is assumed to generate an equal amount of load on each backend task in its subset; this is rarely exactly true in practice but suffices for these purposes.

Utilization is a useful measurement of load balancing: Dividing the total usage by the total capacity gives the fraction of resources being used. This can be applied to the connection distribution: Total usage will be the total number of connections ($M \times k$), and (since we provision for the most loaded backend task) the total capacity will be based on the backend task with the most connections ($\max(C_n) \times N$, where C_n is the number of connections to the n th backend task). This provides the following metric:

$$Utilization = \frac{M \times k}{\max(C_n) \times N} = \frac{\frac{M \times k}{N}}{\max(C_n)} = \frac{\text{mean}(C_n)}{\max(C_n)}$$

This metric, however, does not take into account the discrete nature of connections. If $M \times k$ is not divisible by N , an ideal subsetting algorithm has to assign either $\lfloor \frac{M \times k}{N} \rfloor$ or $\lceil \frac{M \times k}{N} \rceil$ connections to each backend task, so $\max(C_n) = \lceil \frac{M \times k}{N} \rceil$ and $Utilization < 1$. To achieve $Utilization = 1$ in this case, the metric must be adjusted to give the achievable utilization:

$$Utilization = \frac{\lceil \text{mean}(C_n) \rceil}{\max(C_n)} = \frac{\lceil \frac{M \times k}{N} \rceil}{\max(C_n)}$$

Using this metric compares connection balance for subsetting algorithms across a variety of different scenarios. Note that achieving a high utilization is straightforward in two ways. First, increasing k naturally improves utilization because it decreases the effect of subsetting on load balancing; increasing the subset size to N would disable subsetting entirely. Second, as the ratio of frontend tasks to backend tasks increases, the subsetting algorithm has “more choices” per backend task, so the connection balance improves naturally even if choosing randomly. This is shown in Figure 2, which plots utilization against the ratio of frontend tasks to backend tasks for jobs with, at most, 256 tasks ($k = 20$, $1 \leq M \leq 256$, $k \leq N \leq 256$, $M \times k > N$); while not a realistic bound, this is sufficient to demonstrate the algorithm’s behavior.

Round-robin subsetting. Random subsetting can be improved by introducing coordination between the

frontend tasks via their task numbers (0 to $M-1$). Round-robin subsetting assigns backend tasks consecutively to the first frontend task's subset, and then the second task's, and so on, demonstrated in Table 1. Each frontend task m can efficiently generate its subset by starting at backend task number.

It should be fairly straightforward to see that this will balance connections as uniformly as possible: Once a backend task n is assigned a connection, it will not be assigned another connection until all other backend tasks have been assigned connections. Although this algorithm has good connection balance, its other behaviors are undesirable.

Property: Subset diversity. Imagine what would happen if there were more frontend tasks in Table 1. Frontend task 5 would get assigned the next four backend tasks, which are $\{0, 1, 2, 3\}$, but this is the same as the subset for frontend task 0. With 10 backend tasks and four tasks per subset, there are $10 \text{ choose } 4 = 210$ possible subsets that could be assigned to frontend tasks, but this algorithm can assign only five distinct subsets. In the general case, there are $N / \gcd(k, N)$ distinct subsets.

Why does this matter? Imagine one of the frontend tasks is canarying a change that triggers bad behavior (for example, high latency or a crash) in the backend tasks in its subset. This will affect other frontend tasks, but those tasks should be able to retry their requests on other backend tasks. If other frontend tasks have the same subset as the canary frontend task, however, they will share the same fate and will be unable to fail over—or will fail over to the same backend tasks, overloading them.

Deterministic subsetting. Subset diversity can be increased by introducing randomness, but this must be done in a way that maintains the connection balance. This leads to a solution where you shuffle all the backend tasks, assign them to the first few front ends, and then repeat.

For example, for the scenario in Table 1, you could shuffle the backend tasks as $[9, 1, 3, 0, 8, 6, 5, 7, 2, 4]$, and assign subsets $\{9, 1, 3, 0\}$ and $\{8, 6, 5, 7\}$ to the first two frontend tasks. This presents a problem, however, as backend tasks 2 and 4 are unassigned. If these carry over to the subset of the next frontend task, you might get the shuffled backend

tasks $[7, 2, 0, 8, 9, 1, 4, 5, 3, 6]$, but you can't assign backend task 2 to the same frontend task. Attempting to skip over that backend (and use backend task 0) is also problematic, as it introduces a dependency where a frontend task would need to compute *every* previous set of shuffled backend tasks, instead of just the one from which it is assigning tasks.

This is solved by omitting the leftover tasks, which introduces only a small amount of connection imbalance. In this example, the frontend task 2 would use the subset $\{7, 2, 0, 8\}$.

This is the algorithm as previously described in *Site Reliability Engineering, How Google Runs Production Systems*, but one improvement remains that can be made by balancing the leftover tasks in each group. The simplest way to achieve this is by choosing (before shuffling) which tasks will be leftovers in a round-robin fashion. For example, the first group of frontend tasks would choose $\{0, 1\}$ to be leftovers and then shuffle the remaining tasks to get subsets $\{8, 3, 9, 2\}$ and $\{4, 6, 5, 7\}$, and then the second group of frontend tasks would choose $\{2, 3\}$ to be leftovers and shuffle the remaining tasks to get subsets $\{9, 7, 1, 6\}$ and $\{0, 5, 4, 8\}$. This additional balancing ensures that all backend tasks are evenly excluded from consideration, producing a better distribution.

This algorithm provides good connection balance and subset diversity and has performed well in production for more than a decade. Until Auto-pilot made horizontal resizing more frequent, the only major problems observed could be attributed to particularly small subset sizes.

Property: Connection churn. Consider what happens to the frontend tasks' subsets when the backend tasks increase from 10 to 11, as shown in Figure 3, with changes highlighted in red.

Despite this being a minor change in size, there are many changes to the subsets—and one unlucky frontend task (3) is assigned a *completely different* subset. When this backend size change happens, each frontend task will disconnect from the back ends no longer in its subset and connect to the newly added ones. Re-establishing these connections involves multiple network round trips, during which time the following can occur:

- Overloaded back ends, since frontend tasks will have fewer estab-

Table 1. Round robin subsetting.

Frontend task	Subset (N = 10)			
0	0	1	2	3
1	4	5	6	7
2	8	9	0	1
3	2	3	4	5
4	6	7	8	9

lished backend tasks across which to balance the load, and the connection distribution will not be balanced.

- Increased errors or latency for requests, if no established backend tasks are usable for a given frontend task.

This kind of connection churn is caused by changing the size of the backend, so it's called *backend churn*. Subsetting algorithms can also have *frontend churn* (from changing the frontend size) and *subset-size churn* (from changing the subset size).

Ideally, the amount of backend churn should be proportional to the change in backend size. For example, if the backend size doubles, it would be reasonable for half of each frontend task's subset to change. This backend churn should be evenly spread across subsets: It might be OK for half of the backends in every frontend's subset to change, but it's not OK for half of the frontends to have all the backends in their subsets change.

None of the algorithms considered so far has had any frontend churn—and it is particularly undesirable in a subsetting algorithm. Suppose a frontend job is overloaded and additional tasks are added to increase capacity. Frontend churn will cause existing frontend tasks to reconnect to some backend tasks, effectively reducing capacity before the additional tasks have had time to start serving.

Subset-size churn is important if the subset size is dynamically adjusted, such as when it's based on the frontend size, backend size, and/or traffic level. It is easy to see that random subsetting has minimal churn: The subset size is used only to take a prefix of the shuffled list. On the other hand, both round-robin and deterministic subsetting depend on the subset size in a way that results in high subset-size churn.

Property: Subset spread. Another interesting interaction to consider is how new software versions are deployed to Borg jobs. Jobs are typically updated via

rolling restart beginning at task 0 and with a limit on the number of in-flight task restarts. Except for outlier tasks that are slow to restart, this means that a block of consecutively numbered tasks will be unavailable during an update.

Consider the effect on round-robin subsetting: In Table 1, the first frontend task's subset {0, 1, 2, 3} are *also* the first four tasks that would be restarted by this procedure; if the number of in-flight tasks is near the subset size, most frontend tasks' subsets would be entirely unavailable at some point during the update. Random and deterministic subsetting perform better because it is unlikely that any individual subset will have relatively close task numbers, but with enough frontend tasks, it is likely that some will experience this problem.

We have observed this problem in practice; it can be mitigated by reducing the number of in-flight tasks allowed (slowing down the update) or by increasing the subset size (increasing cost). Ideally, the subsetting algorithm would spread out the backend task numbers in each subset so the updates have a consistent and minimal effect on the frontend tasks. There is a tension between subset diversity and subset spread: You want many different subsets for the former, but you want to limit which subsets are acceptable for the latter.

In Search of a New Algorithm

These are the desired properties of the backend subsetting algorithm:

- ▶ Good connection balance
- ▶ High subset diversity
- ▶ No frontend churn
- ▶ Low backend churn
- ▶ Low subset-size churn
- ▶ Good subset spread

Other than the frontend churn, optimal performance is not required for any of these—performance merely needs to be *sufficient* across the board to avoid undesirable behavior.

Consistent subsetting. The starting point is based on consistent hashing:^f Each frontend and each backend is assigned a random position on the unit circle; each frontend determines its subset by selecting the first k backends found by moving clockwise around the circle. Figure 4a shows consistent subsetting



We piloted Rocksteadier Subsetting in the services most affected by backend churn. For one service, this unblocked the adoption of Autopilot, yielding significant resource savings and reducing the frequency of production incidents.



with random positions for frontend tasks (blue squares) and backend tasks (yellow circles). The frontend task 0 moves clockwise and selects the first two backends it sees, giving the subset {3, 2}.

When tasks are added to or removed from the circle, other tasks' positions are unaffected. This dramatically reduces the connection churn: When a backend task is added, each frontend task's subset can have at most one change.

Unfortunately, this algorithm does not do so well at connection balance or subset diversity. There is no coordination between frontend tasks in choosing their subsets, so the connection balance is no better than random subsetting. Subset diversity suffers because the first backend task that the frontend task selects determines the rest of the subset, so, at most, N distinct subsets are possible.

Ringsteady subsetting. How can the connection balance of consistent subsetting be improved? The number of connections for a backend task is proportional to the amount of “free space” before the backend task on the circle, so the connection balance is determined by how evenly spaced the backend tasks are around the circle. Given this insight, you can improve on randomly chosen positions by using a sequence of positions that favors an even distribution: a low-discrepancy sequence.^g

This is possible only because the backend tasks are consecutively numbered, so the n^{th} element in the low-discrepancy sequence can be associated with the n^{th} backend task.

The sequence we chose to use at Google is the binary van der Corput sequence,^h which begins (with the addition of 0 as the zeroth element) as 0, $\frac{1}{2}$, $\frac{1}{4}$, $\frac{3}{4}$, $\frac{1}{8}$, $\frac{5}{8}$, $\frac{3}{8}$, $\frac{7}{8}$. These fractions determine where each node is placed on the circle. As shown in Figure 4b, the first task is placed at the top of the circle, the second halfway around, and so on.

One of the reasons for choosing this sequence is its convenience in computing the positions of elements. For example, to get the position of the fifth element (with an 8-bit word size), you reverse the bits of $5 = 000001012$ to get

^f https://en.wikipedia.org/wiki/Consistent_hashing

^g https://en.wikipedia.org/wiki/Low-discrepancy_sequence

^h https://en.wikipedia.org/wiki/Van_der_Corput_sequence

$101000002 = 160$; then, treating this as a fixed-point number gives the position $\frac{160}{2^8} = \frac{5}{8}$.

So far, this article has addressed only the backend tasks, but the requirements for frontend tasks are identical: If there are vastly more backend tasks than frontend tasks, the frontend tasks should be spaced evenly so their subset selections extend as far as possible without overlapping. Using the same sequence for both frontend and backend tasks results in another convenient property: When $M = N$, every frontend task has a distinct subset (starting at the same-numbered backend task) and achieves ideal connection balance. Figure 4b shows our algorithm in action, which we call Ringsteady Subsetting.

Unlike random and deterministic subsetting where subset spread is left to chance, Ringsteady Subsetting guarantees good subset spread: Consecutively numbered backend tasks are placed far apart from each other on the circle, so a subset of consecutive tasks around the circle will have evenly spread task numbers.

Figure 2c shows that our algorithm achieves lower utilization in some cases than deterministic subsetting (see Figure 2b) but is significantly better than random subsetting (see Figure 2a).

Frontend and backend scaling. Unfortunately, the connection balance of Ringsteady Subsetting has a deficiency shown on the right-hand side of Figure 2c: Frontend tasks outnumber backend tasks, but utilization does not converge toward the ideal—unlike deterministic subsetting in Figure 2b. The low-discrepancy sequence results in the positions for frontend and backend tasks

Figure 3. Connection churn for deterministic subsetting.

Frontend task	Subset (N = 10)				Subset (N = 10)			
0	8	3	9	2	7	5	8	9
1	4	6	5	7	6	10	4	3
2	9	7	1	6	8	10	6	0
3	0	5	4	8	2	1	9	7

being close to—but not exactly—evenly spaced. Imbalance exists only for scenarios with leftover tasks.

Then why not make them all exactly evenly spaced? This would require moving them slightly (compare Figures 4b and 4c), which would introduce a small amount of connection churn but should improve the connection balance. We call this *frontend scaling* when applied to frontend tasks and *backend scaling* when applied to backend tasks.

With both frontend and backend scaling, our algorithm will always achieve ideal connection balance. Unfortunately, frontend scaling makes the positions of frontend tasks dependent on the frontend size, which introduces frontend churn and makes it unsuitable for this use case.

Figure 2d shows the results. Compared with Figure 2c, backend scaling has achieved the goal of improving the connection balance when $M > N$, with only a small degradation when $M < N$. While this still achieves lower utilization in some cases than deterministic subsetting (see Figure 2b), it is deemed sufficient.

Rocksteadier subsetting. Ringsteady with backend scaling has *almost* all of our desired properties. It falls short on subset diversity because it is a derivative of consistent subsetting, so only N

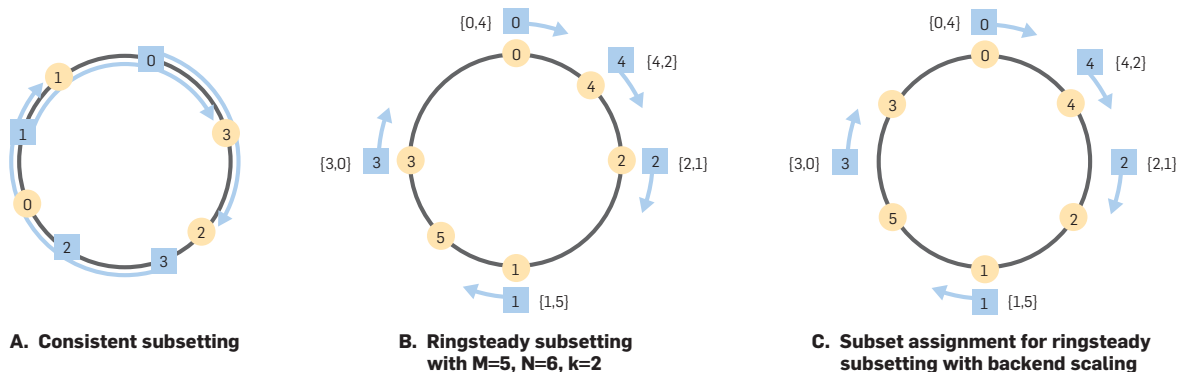
distinct subsets are possible. We investigated Rendezvous Hashingⁱ is an alternative for increasing subset diversity, but it wasn't clear how to improve the connection balance beyond that of random. Instead, we designed an algorithm that incorporates Ringsteady to increase subset diversity without significantly degrading any of the other properties.

Previously, we achieved subset diversity by shuffling every backend task, but this makes the order dependent on the backend size and therefore causes backend churn. Instead, typically the frontend size M is significantly less than the number of possible subsets. Consequently, not *every* backend has to be shuffled (i.e., producing all possible permutations) to achieve sufficient subset diversity.

The way to address this is to form groups (called *lots*) of L backend tasks and shuffle each of those. The parameter L must be a constant: Making it dependent on the frontend size, the backend size, or the subset size introduces connection churn. The last backend lot must consist of L tasks, so if the backend size is not a multiple of L , padding tasks are added; these are not real backend tasks and will be skipped over

i https://en.wikipedia.org/wiki/Rendezvous_hashing

Figure 4. Subsetting.



when choosing subsets. Groups of L frontend tasks are also formed. In each frontend lot, we attempt to distribute the backend tasks uniformly across the frontend tasks—similar to round-robin or deterministic subsetting.

Table 2 shows the first step of this process: grouping frontend and backend tasks into lots using $L = 10$. For illustrative purposes, this is shown for the second frontend lot (tasks 10–19). Since the backend size is not a multiple of L , the

padding tasks 55–59 (indicated in gray text) are added to the last backend lot.

Table 3 shows the second step of this process: shuffling each of the backend lots. The table shows potential subset assignments for frontend tasks 13 (red) and 19 (blue) after shuffling each backend lot from left to right.

The requirements for this process are:

► Each frontend task within a frontend lot needs to use the same

shuffled order for the backend tasks.

► Backend lots should be shuffled differently in different frontend lots.

► Adding a new backend lot must not affect the order of previously shuffled backend lots.

These requirements can be achieved by using the frontend lot number (for a frontend task m , this is $\lfloor m/L \rfloor$) as the seed for a PRNG (pseudorandom number generator) and then using this PRNG to shuffle each of the backend lots in order.

We still need to come up with a way of assigning subsets to frontend tasks. Let's start by considering something simple that doesn't work well. The i^{th} frontend task could look across the i^{th} row and take the backend task from each lot. If we reach the last backend lot in the row, we would wrap around to read from the next row in the table. For example, in Table 3, the frontend task 13 would choose a subset starting with $\{4, 14, 21, 39, 46, 52, 7, 18, \dots\}$. We also skip over padding backend tasks, and the last backend lot wraps around to the first, so frontend task 19 would choose a subset starting with $\{6, 10, 20, 30, 44, 8, 13, 22, \dots\}$.

This method of assigning subsets creates connection imbalance in two ways: It fails to balance across backend lots (such as, over columns); and it fails to balance within backend lots (such as, over rows).

For balancing across backend lots, consider the scenario depicted in Table 3 with a small subset size such as $k = 3$. The subsets for frontend tasks 10–19 would select only backend tasks from the first three columns: Backend tasks 0–29 would each have one connection. Note that this is true for *every* frontend lot: While the order within each backend lot varies by frontend lot, the first backend lot always contains the tasks 0–9, the second 10–19, and so on.

Different frontend lots are needed to reach different backend lots in a way that is evenly distributed and doesn't introduce churn. This part of the problem can be solved by mapping frontend/backend lots in the Rocksteadier algorithm to frontend/backend tasks in Ringsteady. For example, in Figure 4c, frontend task 1 sees the backend tasks in the order $[1, 5, 3, 0, 4, 2]$; in Table 4, the columns are reordered so all the tasks in frontend lot 1 choose backend tasks from lots in that same order. Rocksteadier frontend lot 1

Table 2. Grouping frontend and backend tasks into lots.

Frontend Task (Frontend Lot 1)	Backend Tasks (Backend Lots 0–5)					
	Lot 0	Lot 1	Lot 2	Lot 3	Lot 4	Lot 5
10	0	10	20	30	40	50
11	1	11	21	31	41	51
12	2	12	22	32	42	52
13	3	13	23	33	43	53
14	4	14	24	34	44	54
15	5	15	25	35	45	55
16	6	16	26	36	46	56
17	7	17	27	37	47	57
18	8	18	28	38	48	58
19	9	19	29	39	49	59

Table 3. Potential subset assignments for frontend tasks.

Frontend Task (Frontend Lot 1)	Backend Tasks (Backend Lots 0–5)					
	Lot 0	Lot 1	Lot 2	Lot 3	Lot 4	Lot 5
10	8	13	22	36	42	50
11	3	19	24	37	41	59
12	5	15	26	38	45	55
13	4	14	21	39	46	52
14	7	18	28	31	49	58
15	1	16	25	34	47	51
16	2	11	23	33	40	54
17	9	12	29	32	43	56
18	0	17	27	35	48	53
19	6	10	20	30	44	57

Table 4. Reordering backend lots using ringsteady.

Frontend Task (Frontend Lot 1)	Backend Tasks (Backend Lots 0–5)					
	Lot 1	Lot 5	Lot 3	Lot 0	Lot 4	Lot 2
10	13	50	36	8	42	22
11	19	59	37	3	41	24
12	15	55	38	5	45	26
13	14	52	39	4	46	21
14	18	58	31	7	49	28
15	16	51	34	1	47	25
16	11	54	33	2	40	23
17	12	56	32	9	43	29
18	17	53	35	0	48	27
19	10	57	30	6	44	20

uses an ordering corresponding to Ringsteady frontend task 1.

The remaining (and relatively minor) balancing problem occurs when the last frontend lot is incomplete. For example, consider Table 4 if only frontend tasks 10, 11, and 12 existed with a relatively large subset size: Because their subsets all start on consecutive rows, there will be some overlap between them, resulting in multiple connections to some backend tasks (for example, those on the third row, 5, 45, 26,... may have connections from all three frontend tasks), whereas backend tasks in lower rows will have no connections. This imbalance can be reduced by using a different mapping from the frontend task to the starting row: This is just a fixed permutation of size L ; we arbitrarily choose to use the Ringsteady order [0, 8, 2, 4, 6, 1, 9, 5, 3, 7] to spread consecutive frontend tasks across the rows. Table 5 shows the final subset assignment process, permuting the frontend tasks and showing subset assignments ($k = 10$) for frontend tasks 10 (red), 11 (blue), and 12 (green).

Larger values of L increase subset diversity but at the cost of connection imbalance. Fortunately, relatively small values (such as 10) are able to provide sufficient subset diversity for typical Borg jobs without adding a significant amount of connection imbalance.

Testing and Deployment

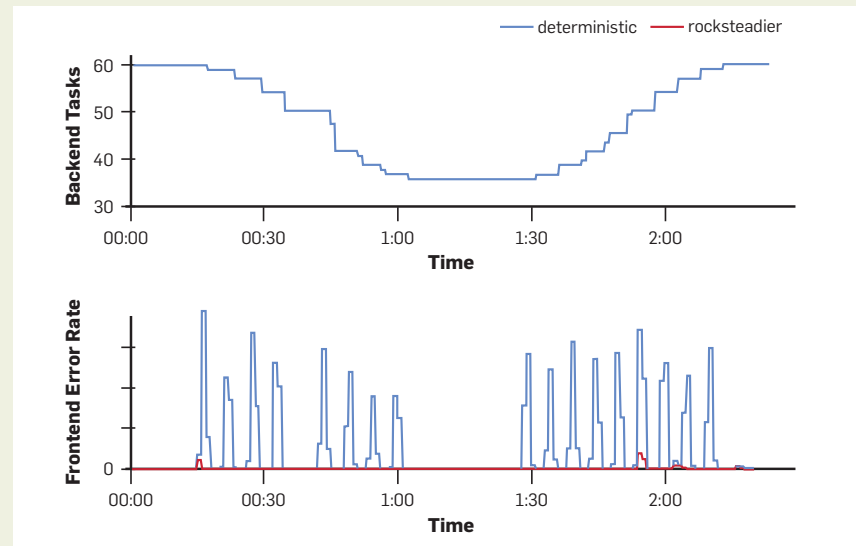
During development, we used a test suite of frontend, backend, and subset sizes gathered from production to compare the properties of different algorithms. This showed that Rocksteadier Subsetting had reduced connection churn, but we wanted to verify that it also reduced the second-order effects we had seen.

To do this, we ran an experiment on a service in a nonproduction environment. Two frontend jobs (one using deterministic subsetting, the other Rocksteadier Subsetting) continuously sent requests to a backend job, which was incrementally resized (with varying step sizes) during the experiment. Figure 5 shows the results: Every time the backend size changed, the frontend job using deterministic subsetting would see a spike of errors, whereas the frontend job using Rocksteadier Subsetting was largely unaffected.

Table 5. Permuting the frontend tasks.

Frontend Task (Frontend Lot 1)	Backend Tasks (Backend Lots 0–5)					
	Lot 1	Lot 5	Lot 3	Lot 0	Lot 4	Lot 2
10	13	50	36	8	42	22
18	19	59	37	3	41	24
12	15	55	38	5	45	26
14	14	52	39	4	46	21
16	18	58	31	7	49	28
11	16	51	34	1	47	25
19	11	54	33	2	40	23
15	12	56	32	9	43	29
13	17	53	35	0	48	27
17	10	57	30	6	44	20

Figure 5. Comparison of backend churn with stepped backend resize.



We piloted Rocksteadier Subsetting in the services most affected by backend churn. For one particular service, this unblocked the adoption of Auto-pilot, yielding significant resource savings and reducing the frequency of production incidents.

After running for some months without any major incidents, Rocksteadier Subsetting was rolled out as the new default backend subsetting algorithm across the fleet. This rollout was successful and went largely unnoticed by service owners.

Conclusion

Google sought an algorithm providing good connection balance, high subset diversity, no frontend churn, low backend churn, low subset-size churn, and good subset spread. Most subsetting algorithms can provide several of

these properties, but to our knowledge, Rocksteadier Subsetting is novel in providing all of them.

Finally, while these trade-offs are appropriate for Google's production environment, they may not be ideal in other environments. Regardless, the discussion of these properties and the explanation of the design process could be useful in other contexts. 

Peter Ward is a senior software engineer in Site Reliability Engineering at Google in Sydney, Australia. He currently works on Google Maps, but has previously worked on various products including Autopilot, Chrome and Google Photos.

Paul Wankadia is a senior software engineer at Google in Sydney, Australia. He retired from Site Reliability Engineering and now works to improve software efficiency.

Kavita Guliani is a technical writer in Site Reliability Engineering at Google, Sunnyvale, CA, USA, partnering with engineers, UX designers, program managers, and clients to create documentation that helps the adoption of a product or service.

Copyright held by owner/author.
Publication rights licensed to ACM.